

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – SHORT ANSWER TEST

Course Code: CSA0305 Course Title: Data Structures
CO Assessed: CO5 – Manipulation (BL3, BL4) Assessment: Assessment Tool 3 – Short Answer Test
Weightage: 30% of CIA Total Marks: 100
CO2 Statement: Construct MST using shortest path algorithms and traverse the graph to model and analyse real world problem.
Student Name: Sumatha Reg. No.: 192521097
Section / Batch: Date: 04/09/2026

Code of Conduct

I Sumatha Sai-S / 192521097 (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sumatha

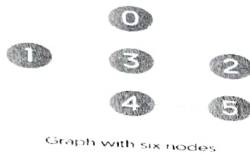
Instructions

- This test contains 10 short answer test questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Minimum Spanning Tree	Kruskal's Algorithm	1	20
Graph Sorting	Topological Sort	2	20
Shortest Path Algorithm	MST & Dijkstra's Algorithm	3	20
Shortest Path Algorithm	Dijkstra's Algorithm	4	20
Minimum Spanning Tree	Prim's or Kruskal's Algorithm	5	20
GRAND TOTAL:			100 Marks

Annexure A - Short Answer Test

- Perform the following operations using stack. Assume the size of the stack is 5 and have a value of 22, 55, 33, 66, 88 in the stack from 0 position to size-1. Now perform the following operations: 1) Invert the elements in the stack, 2) Pop (), 3) Pop (), 3) Pop Construct AVL tree for the following elements. Mention the
- Given a collection of elements 10,20,30,40,50,60,70,80. The Key element to be search 70. Compare how searching is performed in Linear Search and Binary Search. Identify the efficient search method. Justify.
- What is the postfix and prefix forms of the following expression? $A+B*(C-D)/(P-R)$
- Construct AVL tree for the following elements. Mention the Rotations in each step of insertion. 1,2,3,4,5,6,7,8,9,10
- Construct a heap by inserting 10, 15, 12, 23, 21, 1, 6, 43, 34, 56, 78, 23, 45 into a binary heap.
- Construct a B-Tree of Order 3 by inserting numbers 34,26,45,12,87,56,78,22,23,24,65,85,95.
- Perform the merge sort for the following inputs. Also, show the result for each step of iteration. 40,20,70,14,60,61,97,30.
- How the following data set can be sorted by quick sort. Show each step-in sorting. 78,23,31,88,43,55,67,55.
- Perform Open Hashing & Linear Probing for the given set of 0,1,81, 4,64,25,16,36, 9 and 49.
- Consider the following graph & construct BFS.



Graph with six nodes

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Conceptual Understanding	30	Demonstrates complete and accurate understanding of concepts	Good understanding with minor gaps	Basic understanding	Limited understanding
Accuracy of Answer	25	Answers are completely correct and relevant	Mostly correct with minor errors	Partially correct	Mostly incorrect
Problem-Solving & Reasoning	20	Provides logical reasoning and appropriate steps	Good reasoning with minor gaps	Basic reasoning	Lacks logical reasoning
Clarity & Relevance	15	Answers are precise, clear, concise, and directly address the question	Mostly clear and relevant	Somewhat unclear or incomplete	Irrelevant or unclear answers
Time Management & Completion	10	Completes all questions accurately within the allotted time	Completes most questions on time	Requires additional time	Unable to complete the test

Faculty In-charge

Course Coordinator

Program Director

Signature: _____

Signature: _____

Signature: _____

Date: _____

Date: _____

Date: _____

(05 - AT3 Short Answer Test)

1. Stack Operations:

given stack size = 5;

Initial stack:

Position	0	1	2	3	4
Element	22	55	33	66	88

Here, 88 is TOP

Step 1: Invert the stack.

After inversion:

Position	0	1	2	3	4
Element	88	66	33	55	22

Top = 22

Step 2: POP()

22 is removed

Stack: 88 → 66 → 33 → 55

TOP = 55

Step 3: POP()

55 is removed.

Stack: 88 → 66 → 33

TOP = 33

Step 4: POP()

33 is removed

Stack: 88 → 66

Final Answer: Popped elements: 22, 65, 33

Remaining Stack: 88, 66

Data structure used: Stack (LIFO)

2. Linear Search vs Binary Search:

Given:

10, 20, 30, 40, 50, 60, 70, 80.

Search Key = 70

Linear Search : It checks elements one by one.

Comparison	Result.
10 ≠ 70	Continue
20 ≠ 70	Continue
30 ≠ 70	Continue
40 ≠ 70	Continue
50 ≠ 70	Continue
60 ≠ 70	Continue
70 = 70	Found

Comparisons = 7

Time complexity =

⇒ Best case : $O(1)$

⇒ Worst case : $O(n)$

Binary Search : The data is already sorted

$$\text{Mid} = \frac{0+7}{2} = 3$$

Element at index 3 = 40

Since: $70 > 40$

Search the right half: 50, 60, 70, 80.

Middle = 60.

Since, $70 > 60$

Search 70, 80

Middle = 70 → Found

Comparisons = 3

Comparison Feature	Linear Search	Binary Search
Data requirement	Can be unsorted	Must be sorted
Searching method	Sequential	Divide and conquer
Comparisons	7	3
Worst-case complexity	$O(n)$	$O(\log n)$
Efficiency	Lower	Higher

Conclusion:

Binary Search is more efficient for this sorted dataset because it eliminates approximately half of the remaining elements at every step.

3. Postfix and Prefix

Expression:

$$A+B * (C-D) / (P-R)$$

Operator precedence

1. Parentheses
2. Multiplication / Division
3. Addition

Postfix conversion.

$$\text{First: } (C-D) \rightarrow CD-$$

$$(P-R) \rightarrow PR-$$

Then:

$$B * (C-D) \rightarrow BCD-*$$

Therefore:

Postfix

$$*ABCD-PR-/+$$

Prefix conversion:

$$(C-D) \rightarrow -CD$$

$$(P-R) \rightarrow -PR$$

$$B * (C-D) \rightarrow *B-CD$$

Therefore:

Prefix: $A + A/B - CD - PR$

4. AVL Tree — 1 to 10

Insert: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10.

AVL tree maintains:

Balance factor $\Rightarrow$ Height (Left) - Height (Right).

Allowed values: $-1, 0, +1$

Insertion 1.

1, No rotation

Insert 2



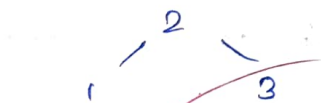
No rotation

Insert 3

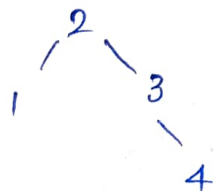
Before balancing: 1-2-3

This is RR imbalance at 1

Rotation: Left Rotation at 1



Insert 4

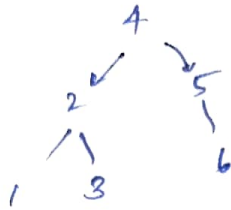


No rotation.

Insert 5

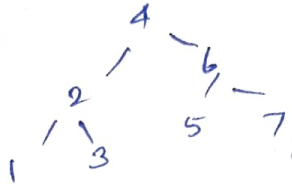
RR imbalance at 3

Left Rotation at 2

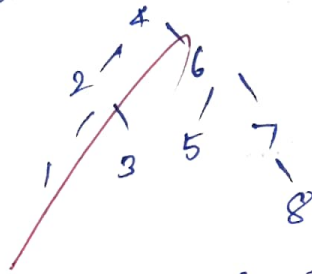


Insert 7 $\Rightarrow$ RR imbalance at 5

Left Rotation at 5



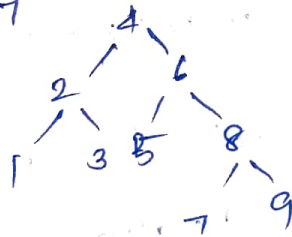
Insert 8



No rotation is required

Insert 9 RR Imbalance at 7

Left Rotation at 7



Insert 10

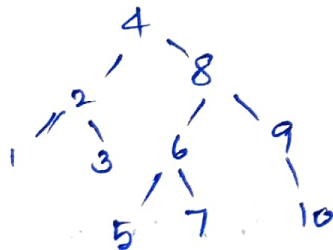
After insertion, imbalance occurs in the

right subtree

left

balance.

Final AVL tree:



Binary Heap:

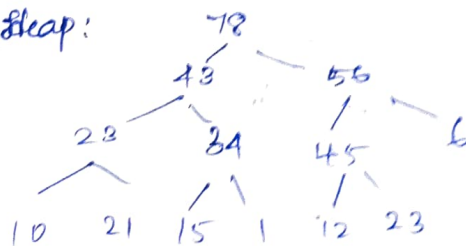
Elements: 10, 15, 12, 23, 21, 1, 6, 43, 34, 56, 78, 22, 45

Assuming a Max Heap every parent must be greater than or equal to its children.

Insertion steps:

Insert	Heap array
10	10
15	15, 10
12	15, 10, 12
23	23, 15, 12, 10
21	23, 21, 12, 10, 15
1	23, 21, 12, 10, 15, 1
6	23, 21, 12, 10, 15, 1, 6
43	43, 23, 12, 21, 10, 15, 1, 6
34	43, 34, 12, 23, 15, 1, 6, 10, 21
56	56, 43, 12, 23, 34, 1, 6, 10, 21, 15
78	78, 43, 56, 23, 34, 12, 6, 10, 21, 15, 1
22	78, 43, 56, 23, 34, 45, 6, 10, 21, 15, 1, 12, 22

Final Max Heap:



Heap property: Every parent $\geq$ its children.

6.

B-Tree of Order 3

Keys: 34, 26, 45, 12, 87, 56, 78, 22, 23, 24, 65, 85, 95

For an order-3 B-tree, a node can have at most:

- * 3 children.

- * 2 keys

Important splitting steps

Insert 34: [34]

Insert 26: [26 | 34]

Insert 45: [34]

~~[26] [45]~~

Insert 12:

~~[34]~~
~~[12 | 26] [45]~~

Insert 87:

[34]

[12 | 26] [45 | 87]

Insert 56 causes split of right child.

[34 | 56]

[12 | 26] [45] [87]

Final B-tree:

[34 | 56]

[22] [26] [78]

[12] [23] [24] [45] [65] [85 | 87 | 95]

Merge Sort

Input: 40, 20, 70, 14, 60, 61, 97, 30

Step 1: Divide

[40, 20, 70, 14, 60, 61, 97, 30]

[40, 20, 70, 14]

[60, 61, 97, 30]

[40, 20] [70, 14]

[60, 61] [97, 30]

40 20 70 14

60 61 97 30

Step 2: Merge

[40] + [20] $\rightarrow$ [20, 40]

[70] + [14] $\rightarrow$ [14, 70]

[60] + [61] $\rightarrow$ [60, 61]

[97] + [30] $\rightarrow$ [30, 97]

Next: [20, 40] + [14, 70] $\rightarrow$ [14, 20, 40, 70]

~~[60, 61] + [30, 97] $\rightarrow$ [30, 60, 61, 97]~~

Final: [14, 20, 40, 70] + [30, 60, 61, 97]

$\rightarrow$ [14, 20, 30, 40, 60, 61, 70, 97]

Final Answer: 14, 20, 30, 40, 60, 61, 70, 97

Time complexity: $O(n \log n)$

8. Quick Sort:

Input: 78, 23, 31, 88, 43, 55, 67, 55

Assume the last element is selected as pivot

Step 1:

Pivot = 55

Elements smaller than 55:

23, 31, 43

Elements greater than 55:

78, 88, 67

After partition:

23 31 43 55 78 88 67 55

Because another 55 exists, its exact position depends on the partition convention.

Sort left subarray

23 31 43

Already sorted

23 31 43

Sort right subarray

78 88 67 55

Pivot = 55

55 78 88 67

Now sort: 78 88 67

Pivot = 67

67 78 88

Final sorted sequence: 23, 31, 43, 55, 55, 67, 78, 88

Average time complexity: $O(n \log n)$

Worst case: $O(n^2)$

9. Open Hashing and linear probing:

Data: 0, 1, 81, 4, 64, 25, 16, 36, 9, 49

Assume hash function:

$$h(K) = K \bmod 10$$

Hash values

Key	Hash value
0	0
1	1
81	1
4	4
64	4
25	5
16	6
36	6
9	9
49	9

Open Hashing / Separate chaining

Index	Values
0	0
1	1 → 81
2	—
3	—
4	4 → 64
5	25
6	16 → 256
7	—
8	—
9	9 → 49

Linear Probing:

On collision, move to the next available location.

Insertion:

- * 0 → index 0
- * 1 → index 1
- * 81 → index 1 occupied → index 2
- * 4 → index 4
- * 64 → index 4 occupied → index 5
- * 25 → index 5 occupied → index 6
- * 16 → Index 6 occupied → index 7

* 26 $\rightarrow$ index 6 occupied $\rightarrow$ 7 occupied $\rightarrow$ index 8

* 9 $\rightarrow$ index 9

* 49 $\rightarrow$ index 9 occupied $\rightarrow$ index 0 occupied $\rightarrow$ index 1 occupied $\rightarrow$ index 2 occupied $\rightarrow$ index 3

Final Linear Probing Table:

Index	Key
0	0
1	1
2	87
3	49
4	4
5	64
6	25
7	16
8	26
9	9

Conclusion: Linear probing resolves collision by searching sequentially for the next empty position.

10. Breadth First Search (BFS)

An exact BFS sequence cannot be determined without the graph.

BFS Algorithm:

BFS uses a Queue (FIFO).

1. Select the starting vertex.
2. Mark it as visited.
3. Insert it into the queue.
4. Remove the front vertex.
5. Visit all its unvisited adjacent vertices.
6. Insert those vertices into the queue.
7. Repeat until the queue becomes empty.

Example format:

If the graph starts from vertex A:

Start - A

Queue - A

Visit A



Insert unvisited adjacent vertices

Queue → B, C

Visit B



Insert its unvisited vertices

Queue → C, D, E

Continue until Queue = empty.

100

- The standard error
- standard error by level
- the standard error
- the standard error

100